OTUS

Kotlin Backend Developer.Professional

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Репозиторий с PostgreSQL

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в TG
OTUS Kotlin-2025-08

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Программа курса

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Выбор проекта	Маркетинг аналитика	Security	Архитек-тура	DevOps	
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Защита					

Окатов Сергей
автор программы, руководитель курса

- кандидат физ.-мат. наук
- 15 лет научной работы в университетах РФ, Европы, США
- 10 лет комм. разработки в стартапах и корпорациях (Сбер, Ланит) в качестве руководителя разработки, архитектора
- Специализация
  - Котлин
  - Большие данные
  - Распределенные системы
  - Архитектура

План полета

- О SQL/PostgreSQL
- Работа с БД в боевом проекте
- Реализация репозитория
- Запуск и тестирование
- Рефлексия

Цели вебинара

После занятия вы:

1. Проектировать работу БД в условиях боевых проектов
2. Сможете работать с PostgreSQL в экосистеме Kotlin
3. Сможете запускать и тестировать PostgreSQL в Docker и TestContainers

Смысл

Зачем вам это уметь

1. SQL – классика, PostgreSQL – самый яркий представитель мира SQL
2. Особенности современных боевых проектов существенно отличаются от того, что преподается в тюториалах и книжках
3. Docker и TestContainers предоставляют прекрасный инструмент для интеграционного и сквозного тестирования

Disclaimer

Disclaimer

Сегодняшние темы

1. *SQL, PostgreSQL*
2. Особенности боевых проектов
3. **Репозиторий с ORM Kotlin Exposed**
4. Репозиторий в Native + **SQL (без ORM)**
5. Правка сторонних библиотек
6. *Запуск PostgreSQL в Gradle*
7. Интеграция репозитория в Ktor
8. Интеграция репозитория в Spring
9. **Интеграционные и e2e тесты**

О SQL

о SQL

В требованиях к курсу - знание SQL

Темы занятия:

1. structured query language — «язык структурированных запросов»
2. назначение – сохранение и извлечение данных в персистентном (долговременном) хранилище
3. дополнительно – вычисление аналитических данных
4. **Зародился в 1970-х в IBM**
5. **Первый стандарт в 1986**
6. *Не смотря на стандартизацию, каждая БД имеет собственный диалект SQL*

Элементы SQL

Элементы SQL:

1. SELECT * FROM table WHERE x=0 GROUP BY y SORT by name HAVING sum(z) > 0
2. INSERT INTO table (a, b, c) VALUES (1,2,3)
3. UPDATE table SET x=x+1 WHERE y=2
4. DELETE FROM table WHERE id=123

Индексы SQL

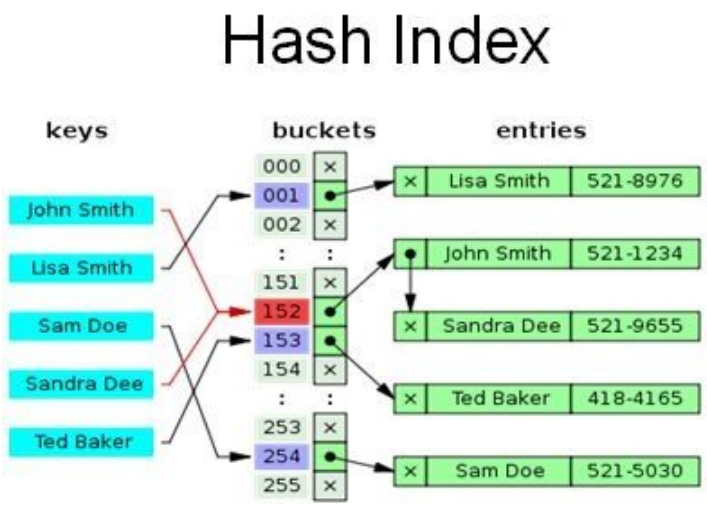
Предназначены для ускорения поиска

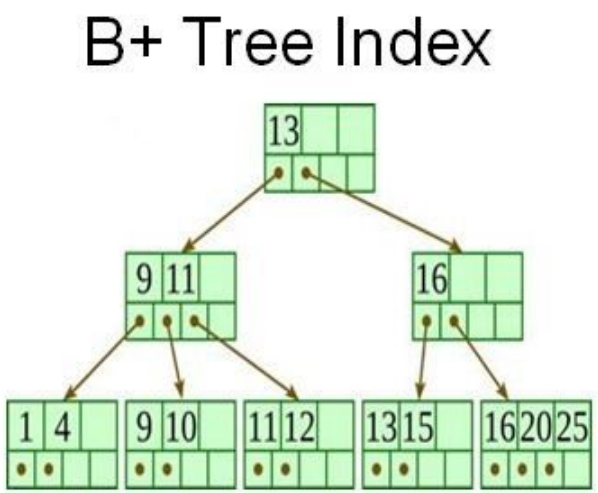
Индексы SQL:

1. BTREE – бинарное дерево – сортировка
2. HASH – без сортировки
3. BITMAP – несколько строк для одного ключа

Типы поисков/индексов:

1. Full Scan – простое сканирование
2. Composite – несколько полей
3. Functional – по формуле
4. Full Text – с разбивкой текста на литералы
5. Primary/Secondary





Транзакции SQL

Обеспечивают пессимистичные блокировки

Транзакции, Уровни:

1. Read Uncommitted – чтение незафиксированных данных
2. Read Committed (default) – видимы только успешно примененные транзакции
3. Repeatable read – повторное чтение в транзакции дает не измененные строки
4. Serializable – повторное чтение в транзакции не содержит новых строк

Dirty Reads – чтение данных транзакции, которая будет отменена

Lost Updates – потеря параллельных UPDATE-ов

Non-repeatable Reads – повторное чтение в рамках транзакции дает другие значения строк

Phantom Reads – повторное чтение в рамках транзакции дает новые вставленные строки

Isolation levels vs read phenomena [edit]

Read phenomena / Isolation level	Dirty reads	Lost updates	Non-repeatable reads	Phantoms
Read Uncommitted	may occur	may occur	may occur	may occur
Read Committed	don't occur	may occur	may occur	may occur
Repeatable Read	don't occur	don't occur	don't occur	may occur
Serializable	don't occur	don't occur	don't occur	don't occur

Расширенные возможности SQL

Расширенные возможности SQL

WITH-запросы:

1. использование одного подзапроса в нескольких местах запроса
2. Разбивка сложных запросов

```
WITH regional sales AS (
    SELECT region, SUM(amount) AS total_sales
    FROM orders
    GROUP BY region
   ), top regions AS (
    SELECT region
    FROM regional sales
    WHERE total_sales > (SELECT SUM(total_sales)/10 F
regional_sales)
   )
SELECT region,
   product,
   SUM(quantity) AS product units,
   SUM(amount) AS product_sales
FROM orders
WHERE region IN (SELECT region FROM top_regions)
GROUP BY region, product;
```

Расширенные возможности SQL

Массивы и JSON/JSONB:

- JSONB - бинарное (сжатое) представление JSON в Postgres
- Индексы по JSONB-колонкам
- Разворачивание и сворачивание JSONB-ячеек
- Поиск по элементам в ячейке
- Конвертация таблица<->массив

```
CREATE TABLE json test (
  id serial primary key,
  data jsonb
);

INSERT INTO json_test (data) VALUES
  ('{}'),
  ('{"a": 1}'),
  ('{"a": 2, "b": ["c", "d"]}'),
  ('{"a": 1, "b": {"c": "d", "e": true}}'),
  ('{"b": 2}');
```

```
SELECT * FROM sal_emp;
 name  |      pay_by_quarter       |                 schedule
-------+---------------------------+-----------------------------------------
 Bill  | {10000,10000,10000,10000} | {{meeting,lunch},{training,presentation}}
 Carol | {20000,25000,25000,25000} | {{breakfast,consulting},{meeting,lunch}}
(2 rows)
```

Расширенные возможности SQL

Fulltext search:

- LIKE ‘%text’ – простое сканирование. Медленно
- Разбивка текста на лексемы tsvector
- Зависит от языка
- Беднее возможностей ElasticSearch/Solr

```
SELECT documents_document.id, ts_rank("text",
plainto_tsquery('запрос'))
FROM documents_document
  LEFT JOIN documents_documentvector ON
documents_document.id =
documents_documentvector.document_id
WHERE "text" @@ plainto_tsquery('запрос')
ORDER BY ts_rank("text", plainto_tsquery('запрос'))
DESC;
```

Расширенные возможности SQL

Window Functions:

- Мощнейший инструментарий для агрегаций и аналитики

```
SELECT
    id,
    section,
    header,
    score,
    row number() OVER (ORDER BY score DESC)  AS rating
FROM news
ORDER BY id;
```

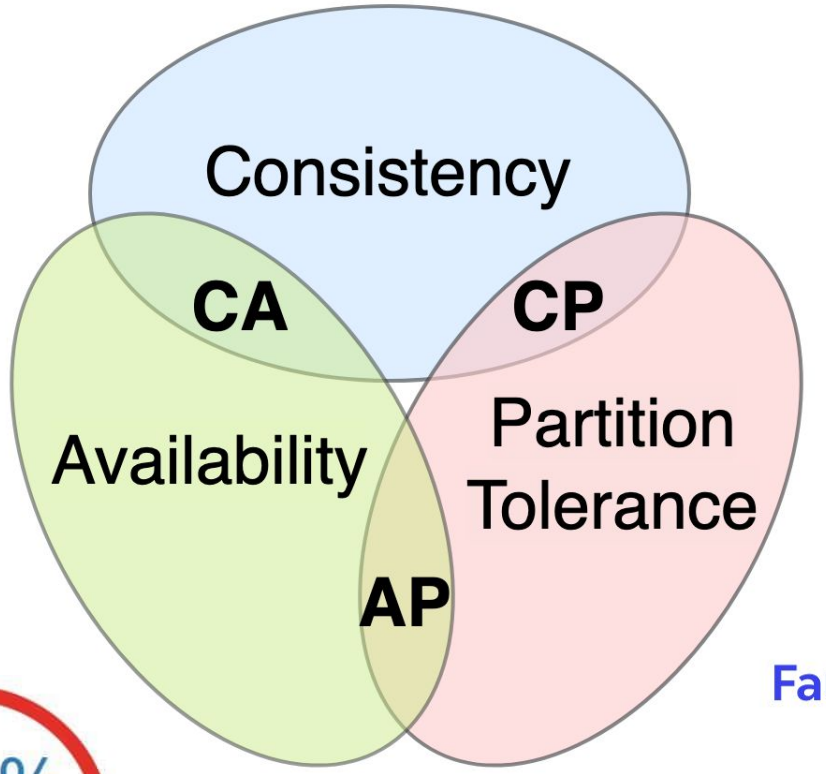
id	section	header	score	rating
1	2	Заголовок	23	7
2	1	Заголовок	6	8
3	4	Заголовок	79	2
4	3	Заголовок	36	4
5	2	Заголовок	34	5
6	2	Заголовок	95	1
7	4	Заголовок	26	6
8	3	Заголовок	36	3

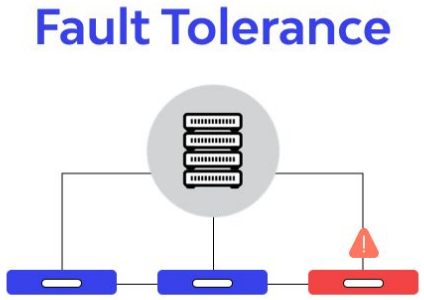
Особенности работы с БД в боевых проектах

Требования бизнеса к работе с данными

Требования:

1. Consistency, Availability (даже при Partitioning)
2. Zero Downtime Deployment
3. Failure Tolerance

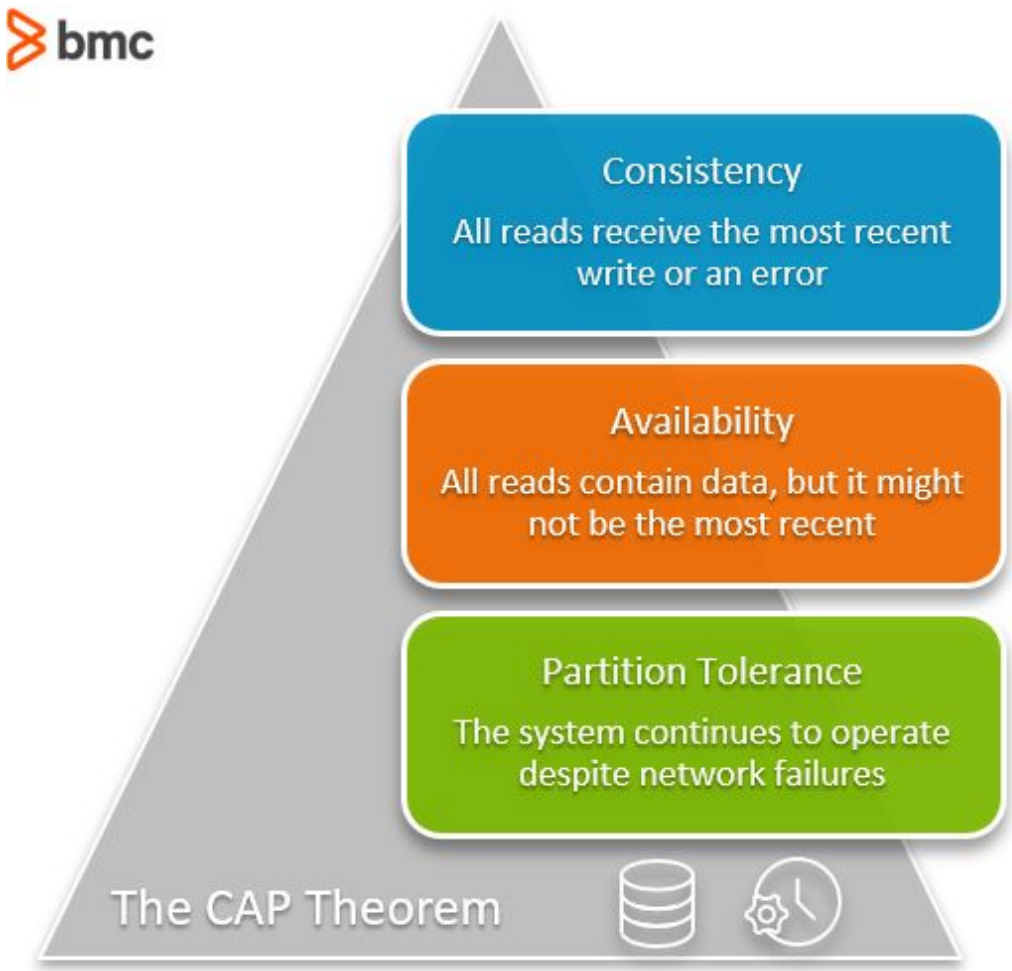




Реалии PostgreSQL (и др.)

Реалии:

1. Легко получить неконсистентность (ошибки БД): мигание света, зависание узла, физические ошибки FS и пр.
2. Нарушение работы из-за логических или человеческих ошибок
3. Снять классический дамп БД - это часы, восстановиться из дампа - часы-сутки



Как работают боевые проекты

Особенности:

1. **Данные первичны**, все программы и все остальное - вторично
2. Кластер из нескольких узлов БД: ускорение, репликация
3. CQRS - command-query responsibility segregation
4. Запрет на изменение схемы из программ
5. Запрет на ручное вмешательство в БД
6. Любые операции с БД только через миграции
7. Online версия БД из прошлого

As your organization's most valuable asset, developing a strategy with data as the primary focus is essential. Without a proper modernization strategy, your data will remain spread across silos and legacy systems, preventing you from capitalizing on its full value and presenting significant roadblocks to increased profitability.

With a data-first modernization approach, you leave data where it resides and rethink infrastructure so processing and storage occur closer to where analysis and decision-making happen. Your organization gains the efficiency and insights that will help maximize productivity and drive revenue growth. You'll gain a cloud operational experience so you can simplify data management with on-demand access, as well as automated data protection and workload placement.

When data is accessible everywhere from edge to cloud, your organization can centralize operations and revamp IT services, delivering capabilities as a service to reduce costs and gain efficiencies. And with simplified data access, you can automate analytics workflows, integrate AI, and scale quickly and intelli which reduces management overhead and optimizes performa and agility.

Clustering

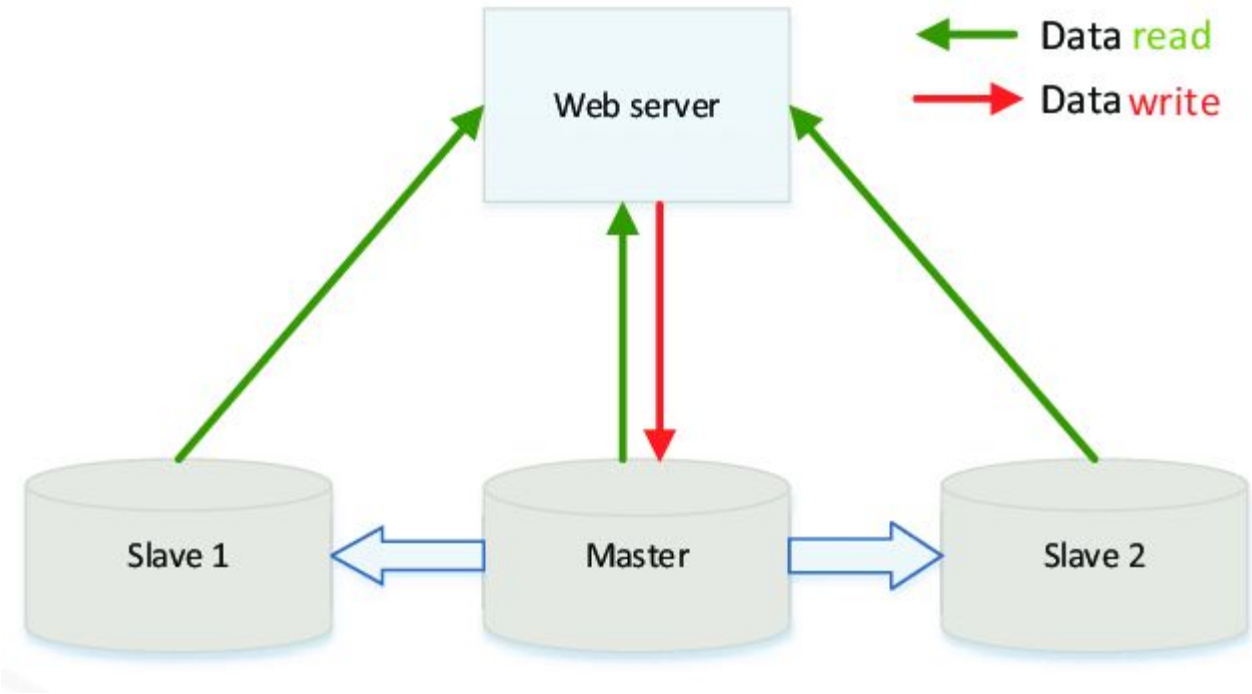
Кластеризация SQL

Шардирования в PostgreSQL нет. Только репликация (CQRS)

Инструменты: pgpool II, pgbouncer, patroni

CRUDS:

1. Create - master
2. Read - slave
3. Update - master
4. Delete - master
5. Search - slave



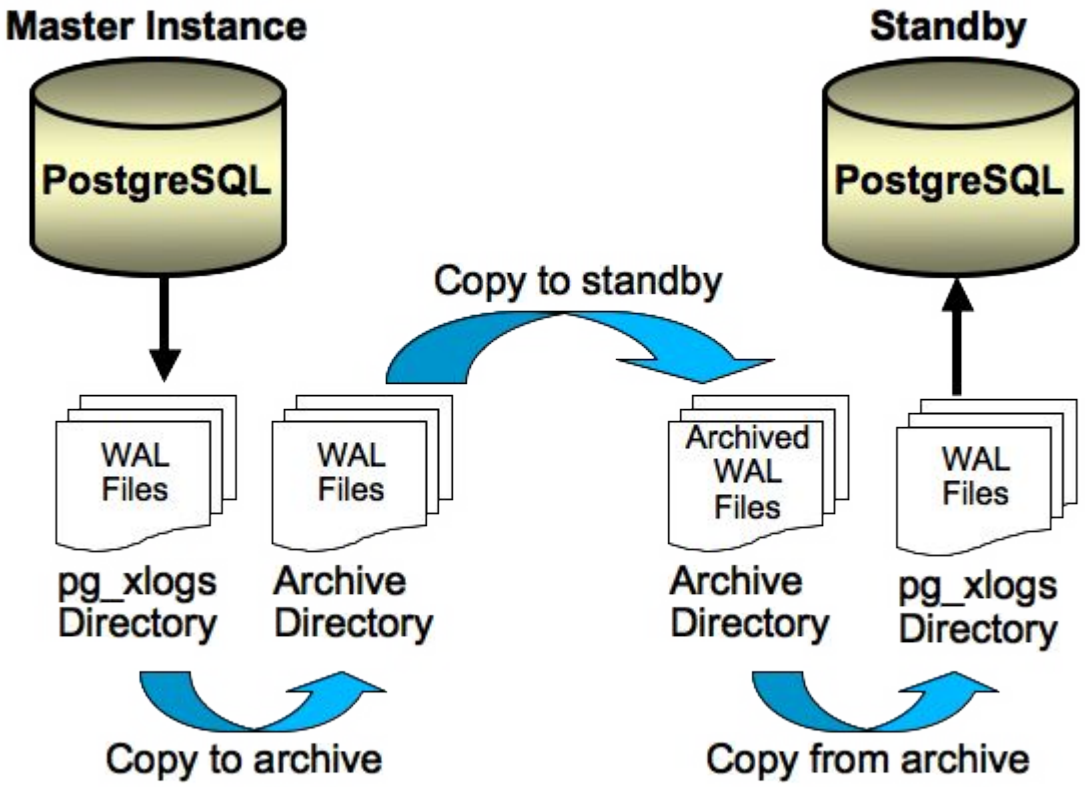
Обеспечение надежности

Методы:

1. *Backup (резервное хранение) RTO – recovery time objective* 😟
2. Continuous Archiving - постоянное копирование WAL
3. Delayed archive - копирование WAL с задержкой

Откат на несколько часов назад - гарантированная потеря данных за несколько часов

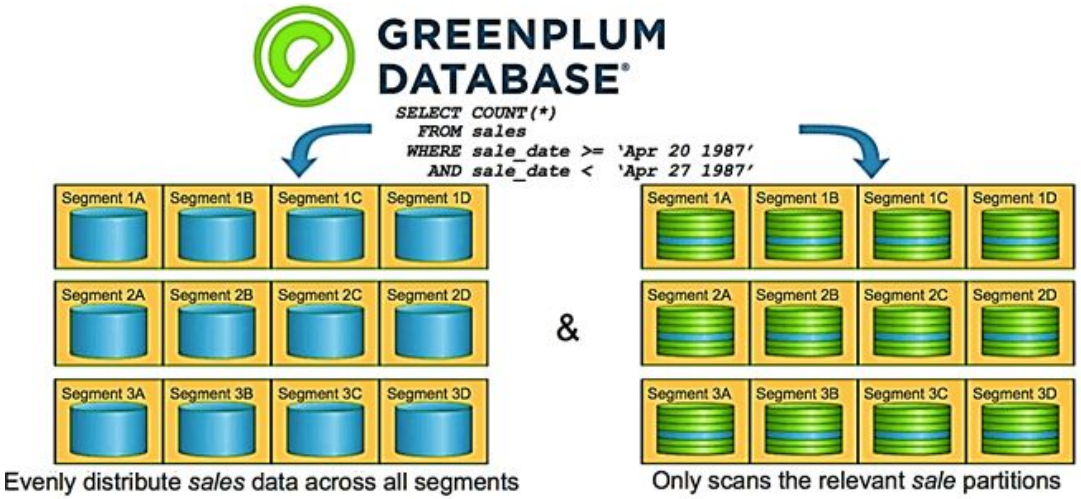
Допускается SLA?



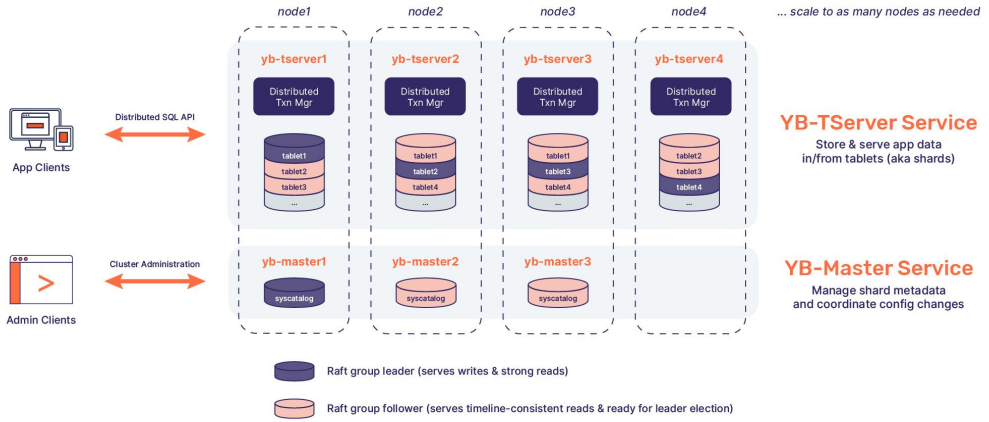
NewSQL альтернативы PostgreSQL

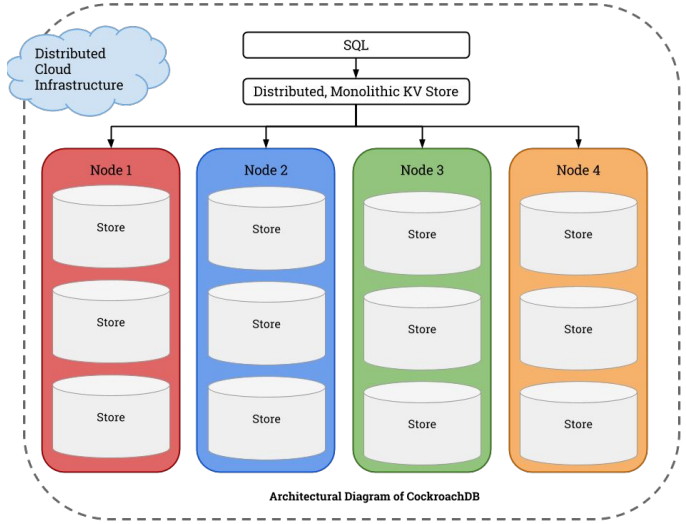
Библиотеки:

- GreenPlum
- NeonDB
- Cockroach
- YugaByteDB



Проблемы совместимости



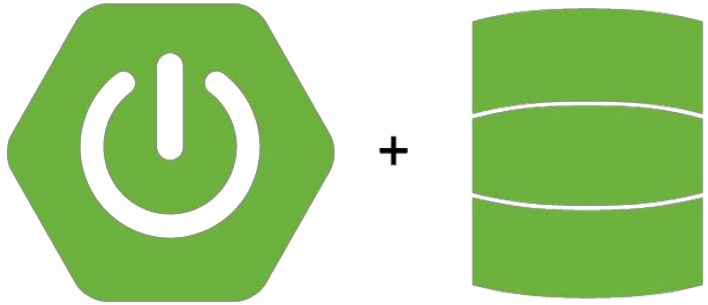


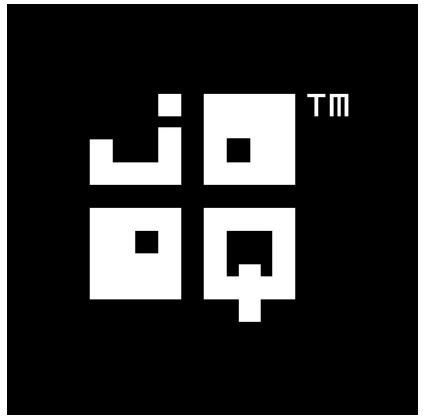
Библиотеки и ORM для PostgreSQL в Kotlin

ORM для PostgreSQL

Библиотеки:

- Hibernate
- Spring Data
- **Kotlin Exposed**
- Kotlin Ktorm
- JOOQ
- MyBatis
- SQLDelight
  - Есть Native, но только для SQLite;
  - PG Native - отдельный проект, не работает
- **pgkn - Kotlin Native PG**





Kotlin exposed

Библиотеки:

- SQL DSL
- DAO
- импорт core, dao, jdbc

```
val exposedVersion: String by project
dependencies {
    implementation("org.jetbrains.exposed:exposed-core:$exposedVersion")
    implementation("org.jetbrains.exposed:exposed-dao:$exposedVersion")
    implementation("org.jetbrains.exposed:exposed-jdbc:$exposedVersion")
  }

 object Users : Table() {
     val id = varchar("id", 10) // Column<String>
     val name = varchar("name", length = 50) // Column<String>
     val cityId = (integer("city_id") references Cities.id).nullable() // Column<Int?>

     override val primaryKey = PrimaryKey(id, name = "PK_User_ID") // name is optional here
   }


    transaction {

        SchemaUtils.create (Cities, Users)

         val saintPetersburgId = Cities.insert {
             it[name] = "St. Petersburg"
         } get Cities.id
    }
```

LIVE

Repository with Exposed

LIVE

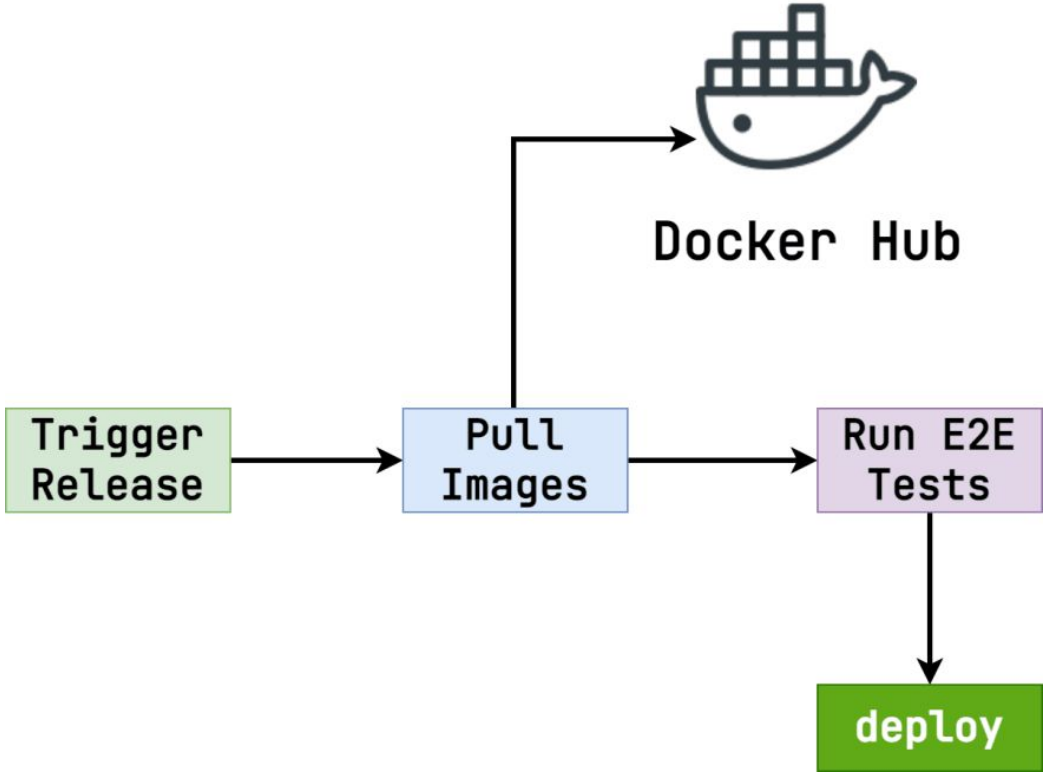
Optimistic Lock in Vanilla SQL

Интеграционное тестирование проекта

Интеграционные и сквозные тесты

О тестах:

1. Обсуждали в теме про тестирование
2. У нас есть готовые тесты на WireMock
3. Решают вопросы внешних миграций в тестах
4. Реализованы с использованием docker-compose.yml, поддерживают всю необходимую инфраструктуру
5. Миграции реализованы в docker-compose.yml в виде отдельного образа, который запускается разово



LIVE

Integration & E2E tests

Рефлексия

Тезисы

Подведем итоги

1. Основные операции: select, update, insert, delete
2. Типы индексов: hash, btree, bitmap; composite, functional
3. Уровни изоляции: uncommitted, committed, repeatable, serializable
4. Расширенные возможности: jsonb/array, with, fulltext, window funcs
5. Библиотеки: Hibernate, Spring Data, Exposed, JOOQ

Домашнее задание

1. Выполнить InMemory реализацию репозитария
2. Сделать автоматические тесты для репозитория
3. Создать персистентное хранилище для выпускного проекта
4. Интегрировать хранилище в выпускной проект

Срок: 17.10.2025

Рефлексия

С какими впечатлениями уходите с вебинара?

Как будете применять на практике то,
что узнали на вебинаре?

Цели вебинара

Проверка достижения целей:

1. Проектировать работу БД в условиях боевых проектов
2. Сможете работать с PostgreSQL в экосистеме Kotlin
3. Сможете запускать и тестировать PostgreSQL в Docker и TestContainers

Список материалов для изучения

1. https://github.com/otuskotlin/202505-ok-marketplace
2. SQLDelight: https://github.com/cashapp/sqldelight
3. JOOQ: https://www.jooq.org/doc/latest/manual/getting-started/jooq-and-kotlin/
4. Hibernate Kotlin: https://habr.com/ru/companies/haulmont/articles/572574/
5. Spring JPA: https://www.baeldung.com/kotlin/jpa
6. Exposed: https://github.com/JetBrains/Exposed
7. Exposed: https://habr.com/ru/companies/otus/articles/555134/
8. Ktorm: https://www.ktorm.org/
9. https://docs.liquibase.com/home.html
10. TestContainers: https://java.testcontainers.org/

Следующий вебинар

10.10.2025

Модуль 7: Работа с базами данных

Репозитарий с Cassandra

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары